

Chapter 5: Architecture and Process Model of the RS-DRL-Based Self-Adaptive System (Alternative 3 - Decomposed Figures)

Thesis Documentation

February 8, 2026

Contents

5.1 Chapter Overview and Architectural Perspective	2
5.1.1 Chapter Scope and Objectives	2
5.1.2 Process-Oriented Organization	2
5.1.3 Architectural Baseline	2
5.2 System-Level Self-Adaptation Loop	3
5.2.1 Managed and Managing System Abstraction	3
5.2.2 Continuous Adaptation Cycle (Stages 1–9)	3
5.2.3 Separation of Runtime Adaptation and Asynchronous Learning	4
5.3 Mapping the Proposed Architecture to the MAPE-K Model	5
5.3.1 Conceptual Mapping to MAPE-K	5
5.3.2 Role of RS-DRL within MAPE-K	5
5.3.3 Temporal Classification	5
5.4 Runtime Monitoring and State Construction	6
5.4.1 Runtime Data Collection	7
5.4.2 Data Preprocessing and Feature Extraction	7
5.4.3 State Vector Construction and Validation	8
5.4.4 Process Boundary	9
5.5 Runtime Analysis and Adaptation Triggering	9
5.5.1 Goal Evaluation	9
5.5.2 Adaptation Triggering	9
5.6 Runtime Planning and Adaptation Option Prediction	10
5.6.1 Model Retrieval	10
5.6.2 Adaptation Option Evaluation	11
5.6.3 Pure Inference at Runtime	12
5.7 Adaptation Option Verification	12
5.7.1 Runtime Model Parameterization	12
5.7.2 Statistical Model Checking	13
5.7.3 Quality Evaluation and Decision Justification	14
5.8 Execution and System Reconfiguration	15
5.8.1 Adaptation Option Retrieval	15
5.8.2 Adaptation Application	15
5.8.3 Loop Closure	16

5.9	RS-DRL Feedback and Learning Overview	16
5.9.1	Role of Feedback	16
5.9.2	Reward Generation from Verification	16
5.9.3	Separation Between Feedback Generation and Learning	16
5.10	Asynchronous RS-DRL Learning and Policy Update	16
5.10.1	Learning Lifecycle Overview	17
5.10.2	Experience Collection	17
5.10.3	Novel Reward-Shaping Mechanism	17
5.10.4	Model Training and Storage	18
5.11	Knowledge Management and Process Coordination	19
5.11.1	Knowledge as Shared System Memory	19
5.11.2	Read-Write Coordination Contracts	19
5.11.3	Coordination of Heterogeneous Processes	19
5.12	Chapter Summary	20
5.12.1	Hierarchical Process Decomposition	20
5.12.2	Architectural Contributions	20
5.12.3	Figure Organization Principle	20
5.12.4	Transition to Evaluation	20

5.1 Chapter Overview and Architectural Perspective

5.1.1 Chapter Scope and Objectives

This chapter presents the architecture and process model of the RS-DRL-based self-adaptive system, focusing on system-level organization, runtime control flow, and component interaction patterns. The chapter emphasizes architectural responsibility and design rationale, maintaining clear separation between system architecture (presented here) and algorithmic mechanisms (detailed in appendices).

5.1.2 Process-Oriented Organization

The primary organizing principle is **process progression**—each section answers: “What happens next in the system, and why?” MAPE-K serves as an **explanatory lens**, not a competing organizational structure. This chapter uses **decomposed figures** aligned with section structure, where each figure focuses on one conceptual responsibility. Complete end-to-end workflows are provided in Appendix A for readers requiring full process context.

5.1.3 Architectural Baseline

The architectural baseline is the RS-DRL integration architecture (Figure 4 of the original article), shown in Figure 1.

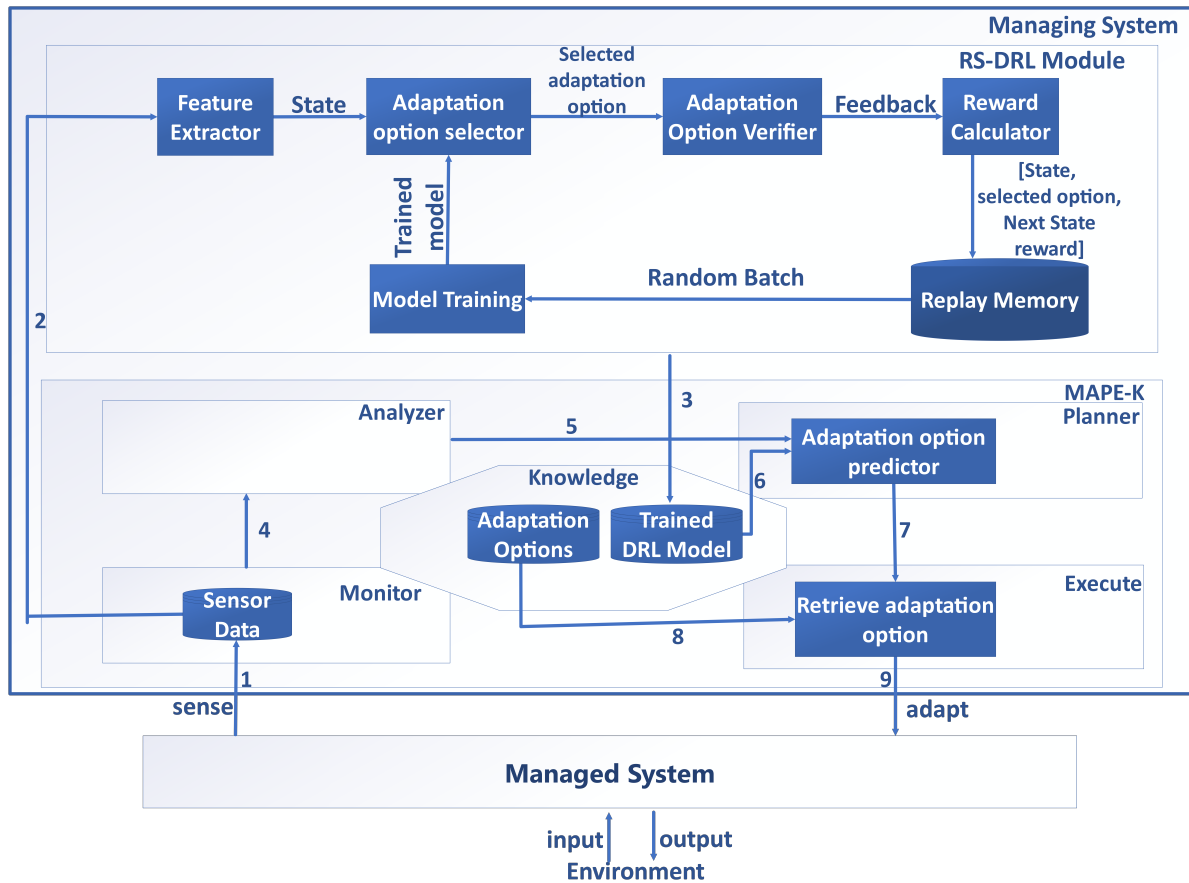


Figure 1: RS-DRL integration architecture showing the integration of RS-DRL with MAPE-K framework.

5.2 System-Level Self-Adaptation Loop

5.2.1 Managed and Managing System Abstraction

The RS-DRL-based self-adaptive system consists of two primary components: the **Managed System** (executes application logic) and the **Managing System** (implements adaptation logic using MAPE-K enhanced with RS-DRL).

5.2.2 Continuous Adaptation Cycle (Stages 1–9)

The system operates through a nine-stage adaptation flow:

1. **Stage 1:** Runtime data collection
2. **Stage 2:** State forwarding to RS-DRL module
3. **Stage 3:** Asynchronous model training and storage
4. **Stage 4:** State forwarding to Analyzer
5. **Stage 5:** Adaptation triggering
6. **Stage 6:** Trained model retrieval
7. **Stage 7:** Adaptation option prediction
8. **Stage 8:** Adaptation option retrieval
9. **Stage 9:** Adaptation application

All subsequent sections refine **subsets of Stages 1–9**, progressively decomposing the system-level loop into focused subprocesses.

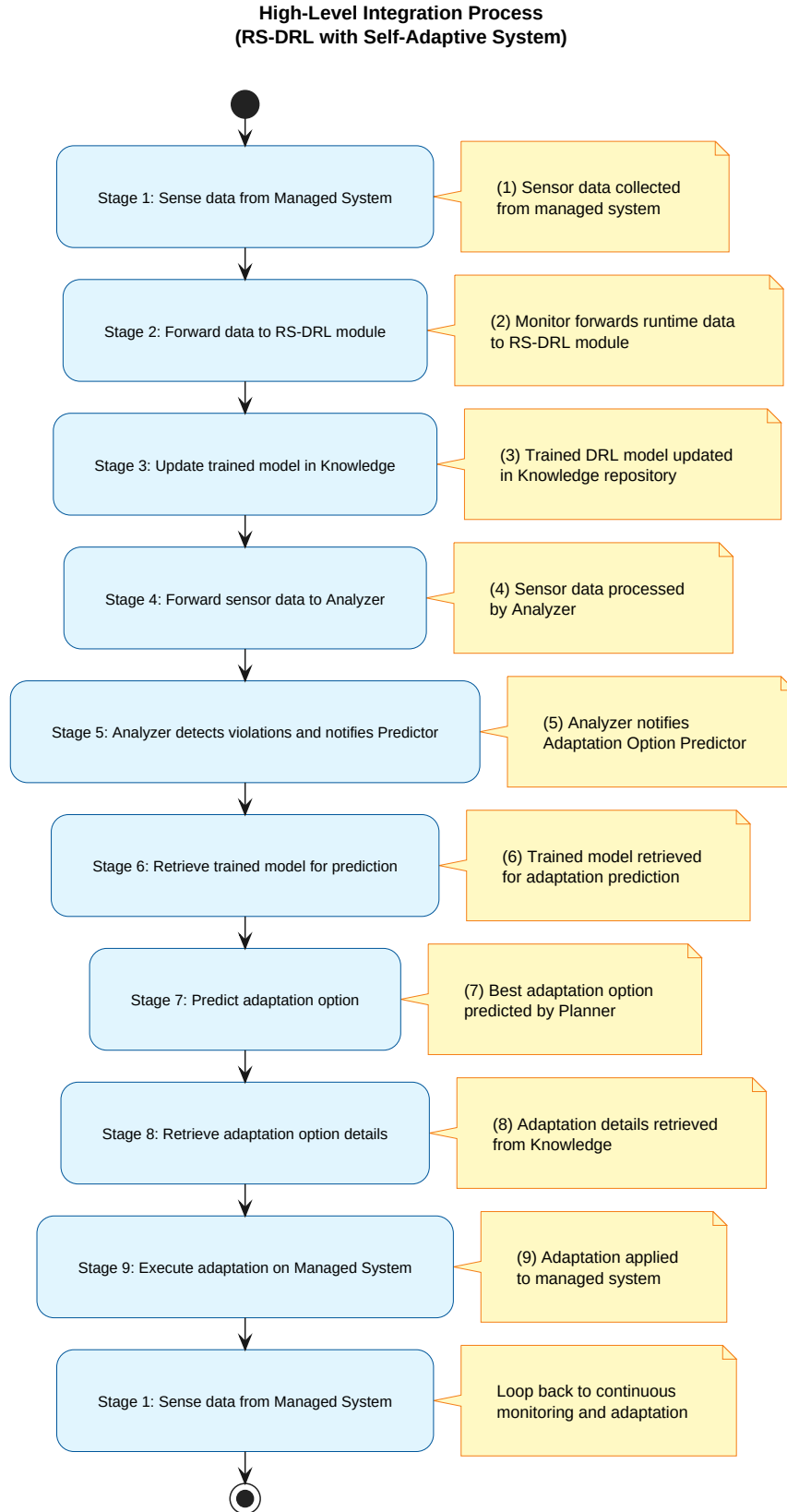


Figure 2: System-level self-adaptation process overview showing the nine-stage adaptation cycle. This high-level view is decomposed into focused subprocesses in subsequent sections.

5.2.3 Separation of Runtime Adaptation and Asynchronous Learning

A critical architectural design decision is the temporal separation between runtime adaptation (Stages 1, 4–9) and asynchronous learning (Stages 2–3). This separation ensures runtime stability while enabling continuous policy improvement.

5.3 Mapping the Proposed Architecture to the MAPE-K Model

5.3.1 Conceptual Mapping to MAPE-K

This section provides a **conceptual mapping** of the RS-DRL architecture to MAPE-K. This mapping serves as a **classification lens**, categorizing the stages according to MAPE-K functional roles.

The mapping is:

- **Monitor** (Stages 1, 2, 4): Observes managed system and constructs state representations
- **Analyze** (Stages 4, 5): Evaluates state against goals and triggers adaptation
- **Plan** (Stages 5, 6, 7): Selects adaptation options using RS-DRL policies
- **Execute** (Stages 8, 9): Applies selected adaptations
- **Knowledge** (Stages 3, 6, 8): Stores models, options, and state history

5.3.2 Role of RS-DRL within MAPE-K

RS-DRL is positioned as an enhancement to the **Plan** phase, providing learned intelligence for adaptation option selection. The RS-DRL module is an **internal decision service within the Planner**, not the system architecture itself.

5.3.3 Temporal Classification

Processes are classified by temporal characteristics:

Process	Temporal Nature
Monitoring	Continuous
Analysis	Event-driven
Planning	Event-driven
Execution	Event-driven
Learning	Asynchronous

Table 1: Temporal classification of architectural processes.

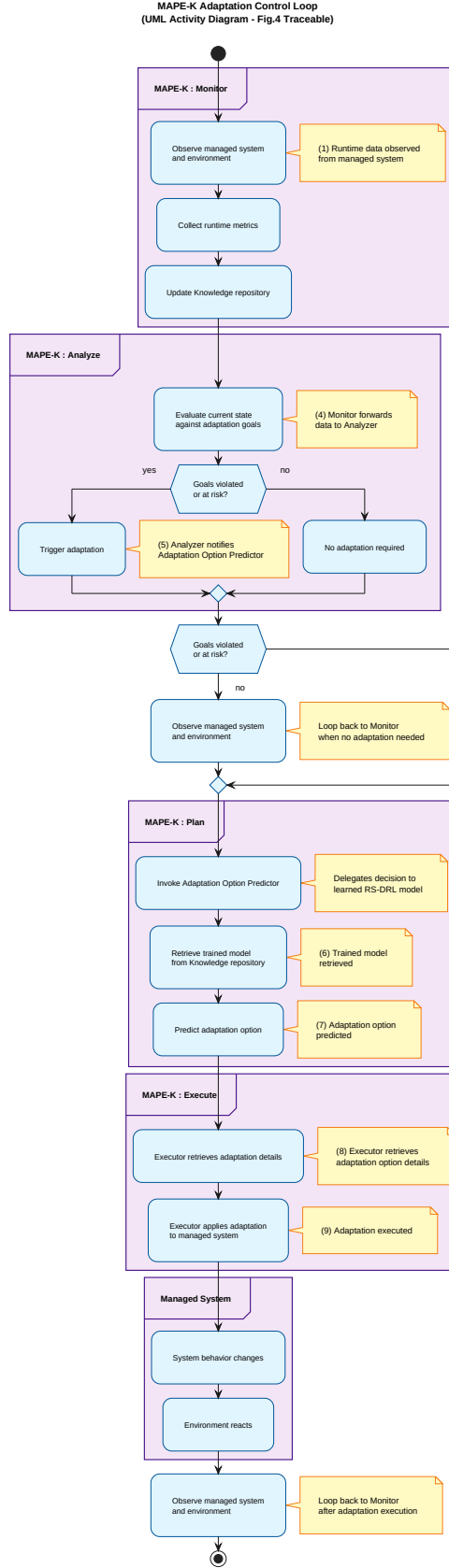


Figure 3: MAPE-K validation view showing RS-DRL enhancement of the planning phase.

5.4 Runtime Monitoring and State Construction

This section describes the monitoring process using **decomposed figures** that align with subsection structure. The complete end-to-end monitoring workflow is provided in Appendix A.1.

5.4.1 Runtime Data Collection

The monitoring process begins with runtime data collection from the managed system and environment (Stage 1).

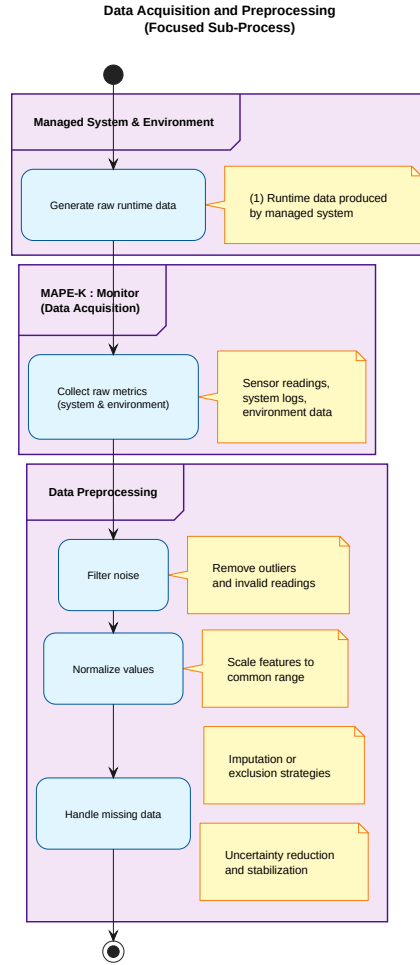


Figure 4: Runtime data acquisition from managed system. This focused view shows data collection only (Stage 1). Complete monitoring workflow in Appendix A.1.

5.4.2 Data Preprocessing and Feature Extraction

Raw data is transformed through preprocessing (filtering, normalization) and feature extraction, deriving goal-relevant attributes while reducing dimensionality.

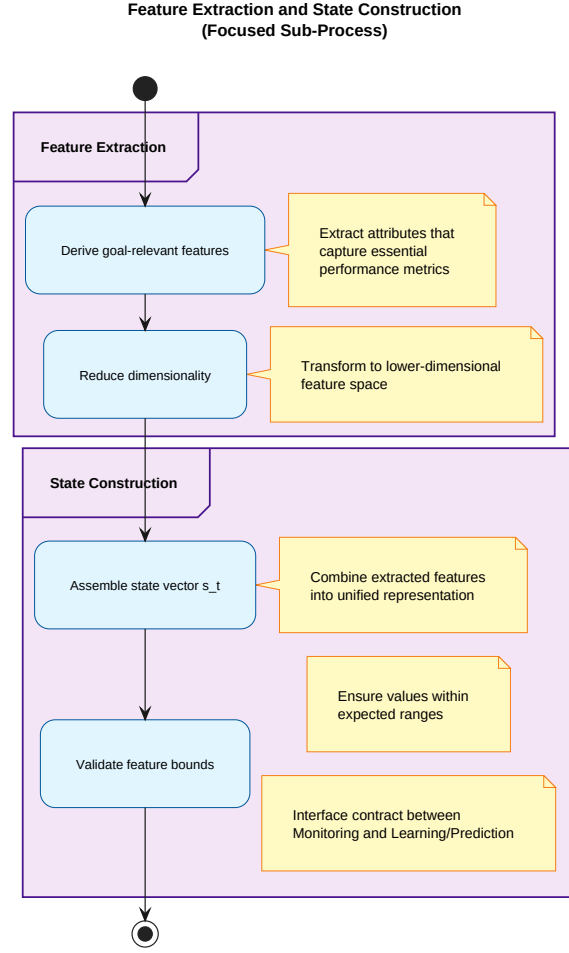


Figure 5: Data preprocessing and feature extraction transforming raw data into feature vectors. Complete monitoring workflow in Appendix A.1.

5.4.3 State Vector Construction and Validation

Extracted features are assembled into a state vector $s_t \in \mathbb{R}^d$ that serves as the interface between monitoring and decision-making. The state vector is validated to ensure consistency.

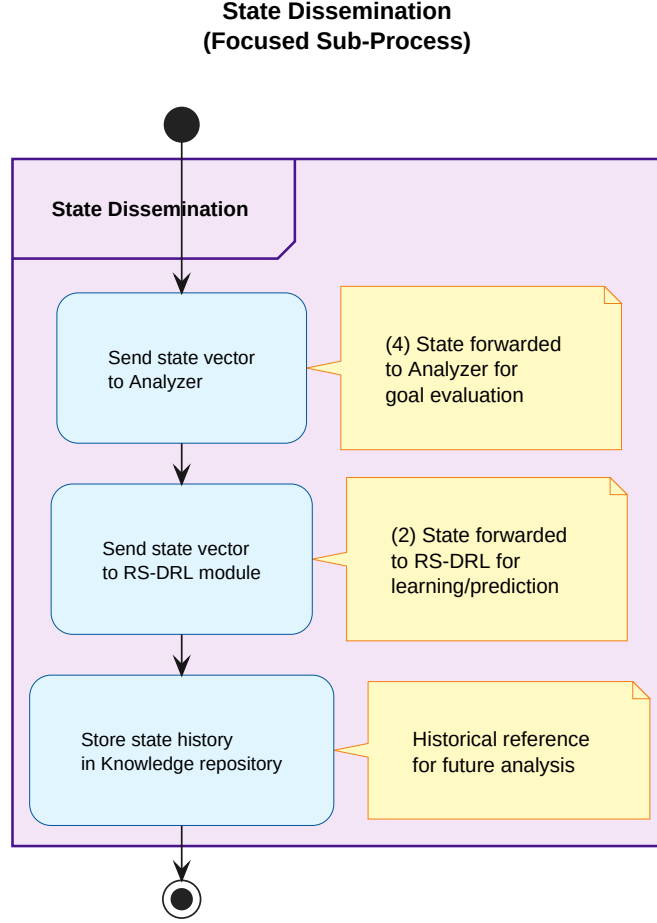


Figure 6: State vector construction, validation, and dissemination to downstream components. Complete monitoring workflow in Appendix A.1.

5.4.4 Process Boundary

Process boundary: The monitoring process ends once a validated state vector exists and has been disseminated. Everything that follows operates on this validated state representation.

For the complete end-to-end monitoring and state construction workflow, see Appendix A.1.

5.5 Runtime Analysis and Adaptation Triggering

5.5.1 Goal Evaluation

The Analyzer receives state vectors from the Monitor (Stage 4) and evaluates whether the current system state satisfies adaptation goals.

5.5.2 Adaptation Triggering

When the Analyzer detects that adaptation goals are violated or at risk, it triggers the planning phase (Stage 5).

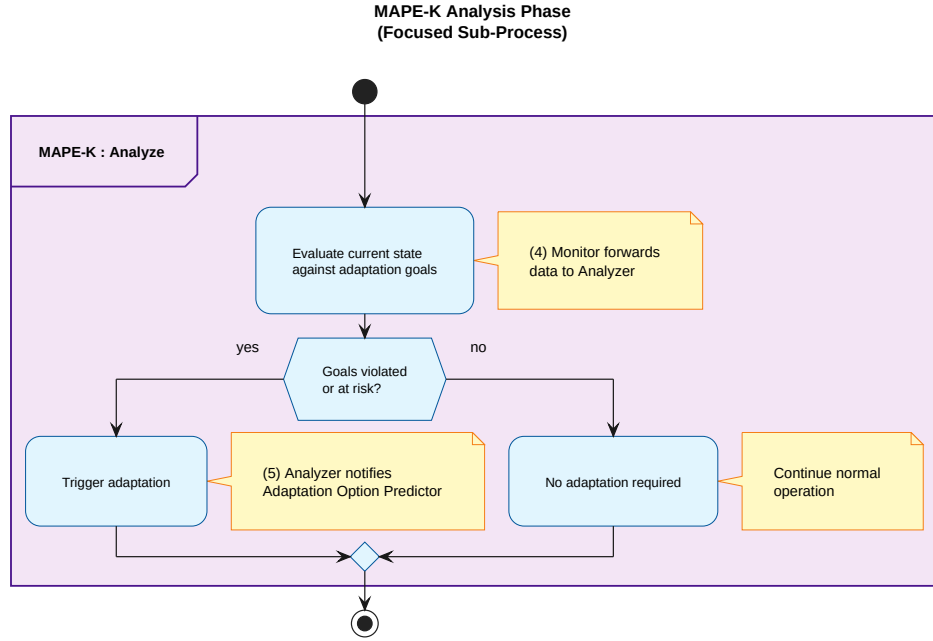


Figure 7: Analysis and adaptation triggering (Stages 4–5). This focused view shows goal evaluation and triggering only.

5.6 Runtime Planning and Adaptation Option Prediction

This section describes the planning process using decomposed figures. The complete adaptation decision workflow is provided in Appendix A.2.

5.6.1 Model Retrieval

Upon receiving the adaptation trigger (Stage 5), the planning process retrieves the trained RS-DRL model from the Knowledge repository (Stage 6).

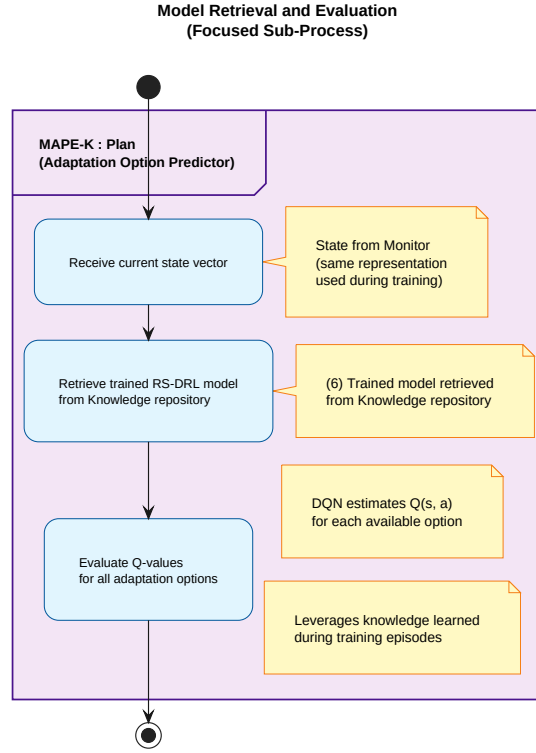


Figure 8: Trained model retrieval from Knowledge repository (Stage 6). Complete decision workflow in Appendix A.2.

5.6.2 Adaptation Option Evaluation

The Adaptation Option Predictor evaluates Q-values for all available adaptation options using the retrieved model.

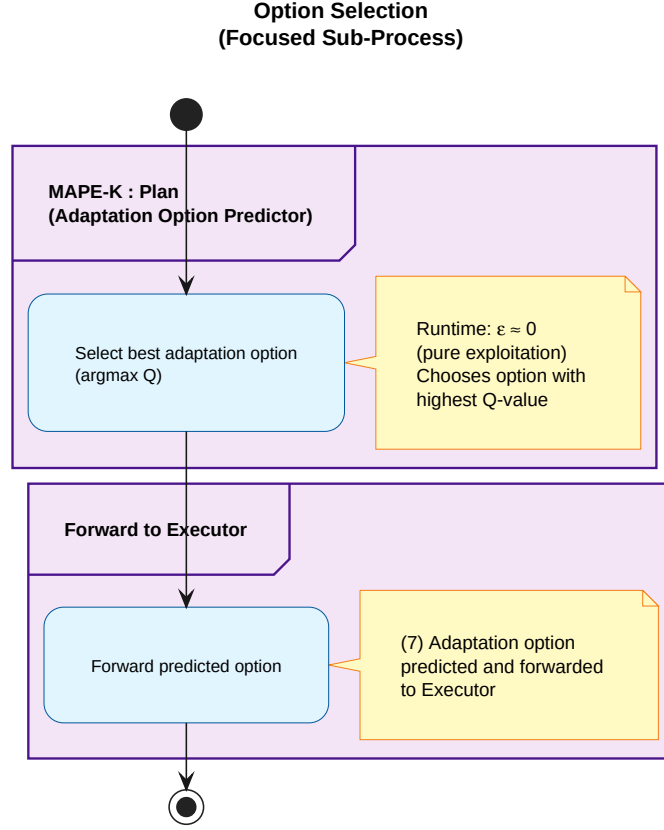


Figure 9: Adaptation option evaluation and selection using learned Q-values (Stage 7). This focused view shows decision-making only. Complete decision workflow in Appendix A.2.

5.6.3 Pure Inference at Runtime

Process boundary: This planning process is **pure inference**—it uses a pre-trained model to evaluate options but performs no learning, no replay memory sampling, and no policy updates.

5.7 Adaptation Option Verification

5.7.1 Runtime Model Parameterization

The predicted adaptation option undergoes verification using runtime models parameterized with current state and candidate option.

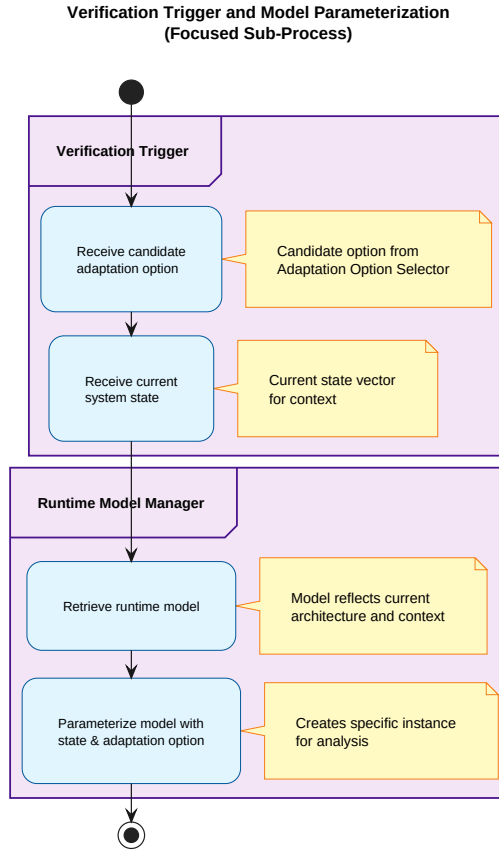


Figure 10: Verification initiation and runtime model parameterization. Complete verification workflow in Appendix B.

5.7.2 Statistical Model Checking

Verification employs statistical model checking (UPPAAL-SMC) to estimate goal satisfaction probabilities under uncertainty.

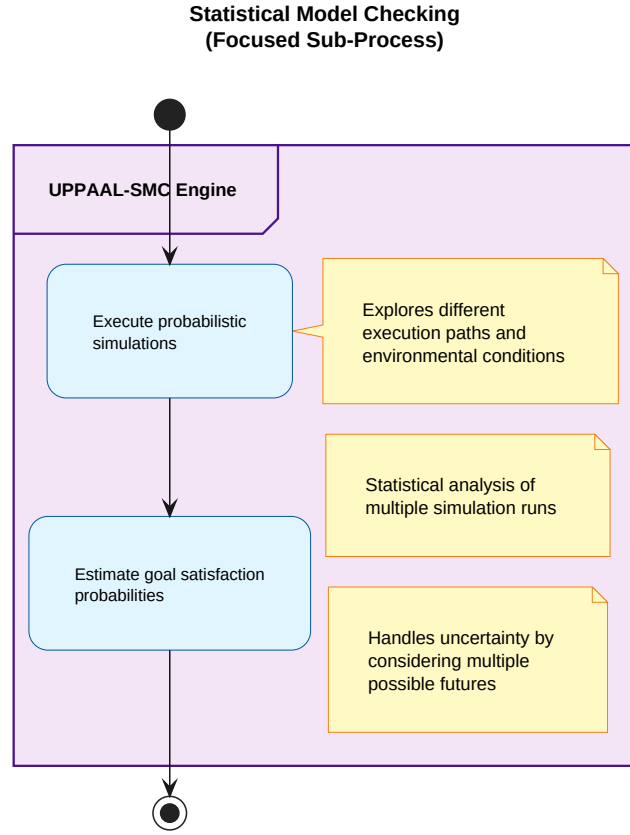


Figure 11: Statistical model checking execution using UPPAAL-SMC. Complete verification workflow in Appendix B.

5.7.3 Quality Evaluation and Decision Justification

Verification generates quality estimates that justify adaptation decisions. During runtime, verification provides **decision justification**. During learning, it generates **reward signals** as a by-product.

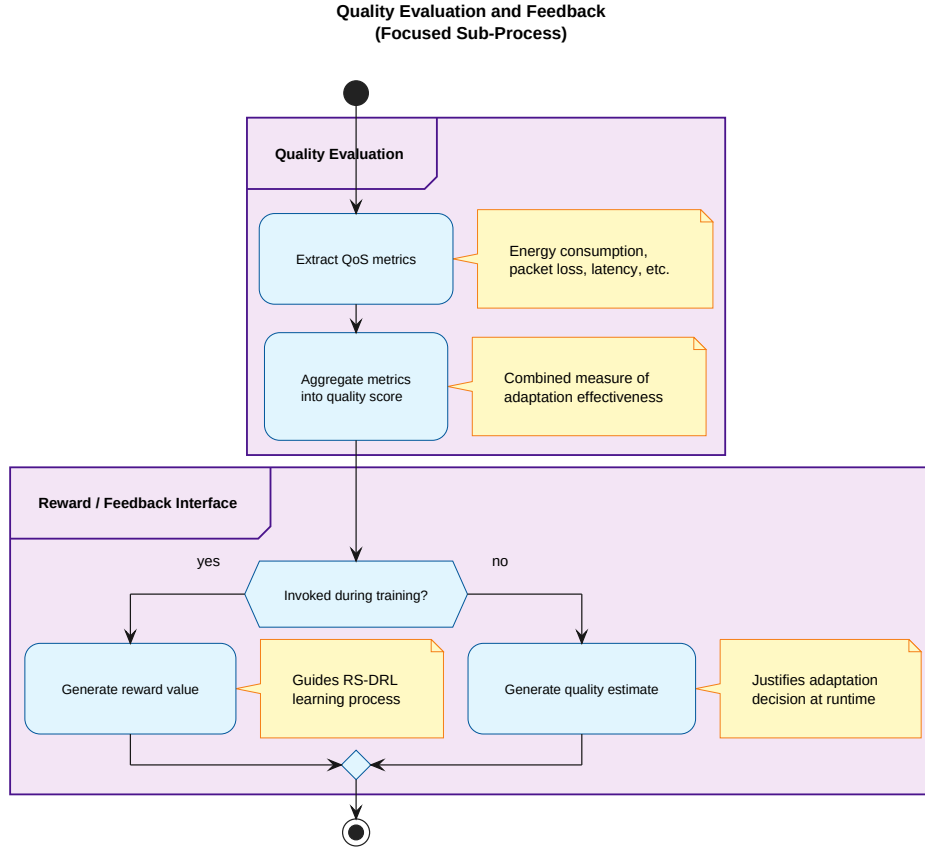


Figure 12: Quality evaluation and feedback generation from verification results. Produces decision justification (runtime) and learning rewards (asynchronous).

For complete verification workflow including all steps, see Appendix B.

5.8 Execution and System Reconfiguration

5.8.1 Adaptation Option Retrieval

The Executor retrieves detailed specifications for the selected adaptation option from the Knowledge repository (Stage 8).

5.8.2 Adaptation Application

The Executor translates the adaptation specification into concrete system modifications (Stage 9), applying configuration changes to the managed system.

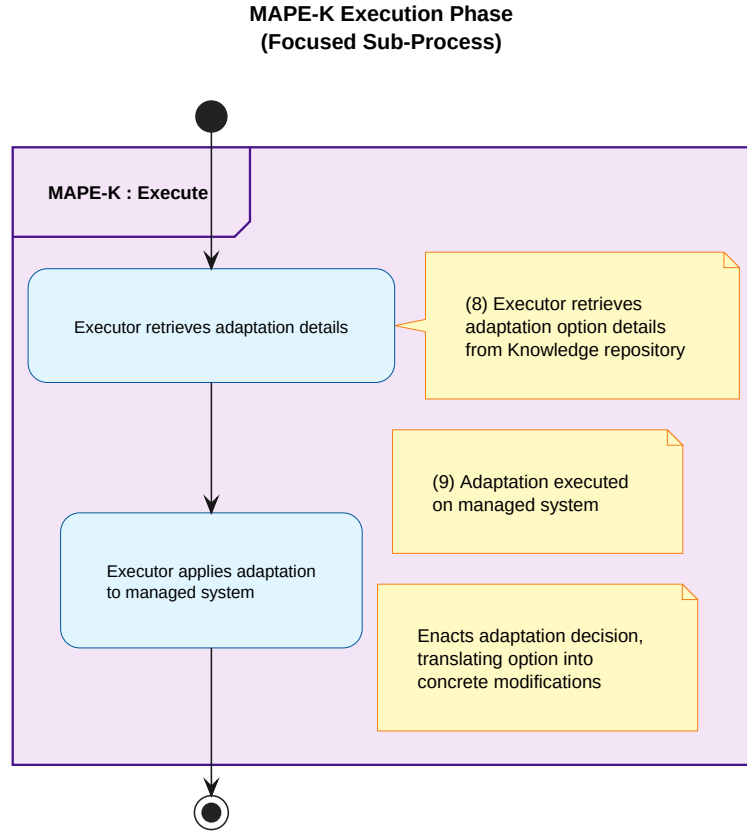


Figure 13: Execution and system reconfiguration (Stages 8–9). This focused view shows adaptation enactment only.

5.8.3 Loop Closure

After adaptation execution, monitoring resumes (Stage 1), closing the adaptation loop and ensuring continuous operation.

5.9 RS-DRL Feedback and Learning Overview

5.9.1 Role of Feedback

Feedback serves as the bridge between runtime adaptation and learning, enabling the system to improve policies based on adaptation outcomes.

5.9.2 Reward Generation from Verification

Reward signals are generated from verification outcomes, capturing quality-of-service metrics and goal satisfaction indicators. These rewards guide the learning process toward policies that achieve adaptation goals.

5.9.3 Separation Between Feedback Generation and Learning

The architecture maintains clear separation between:

- **Feedback generation:** Transforms outcomes into learning signals
- **Policy learning:** Consumes signals to update models (Section 5.10)

5.10 Asynchronous RS-DRL Learning and Policy Update

This section provides an overview of the RS-DRL learning process using decomposed figures. The complete learning workflow is provided in Appendix C.

5.10.1 Learning Lifecycle Overview

The learning lifecycle operates asynchronously with runtime adaptation:

1. Experience collection
2. Experience storage in replay memory
3. Model training with sampled experiences
4. Model storage for runtime use (Stage 3)

5.10.2 Experience Collection

During training episodes, the system collects experiences through environment interaction, generating experience tuples $\langle s_t, a_t, r_t, s_{t+1} \rangle$.

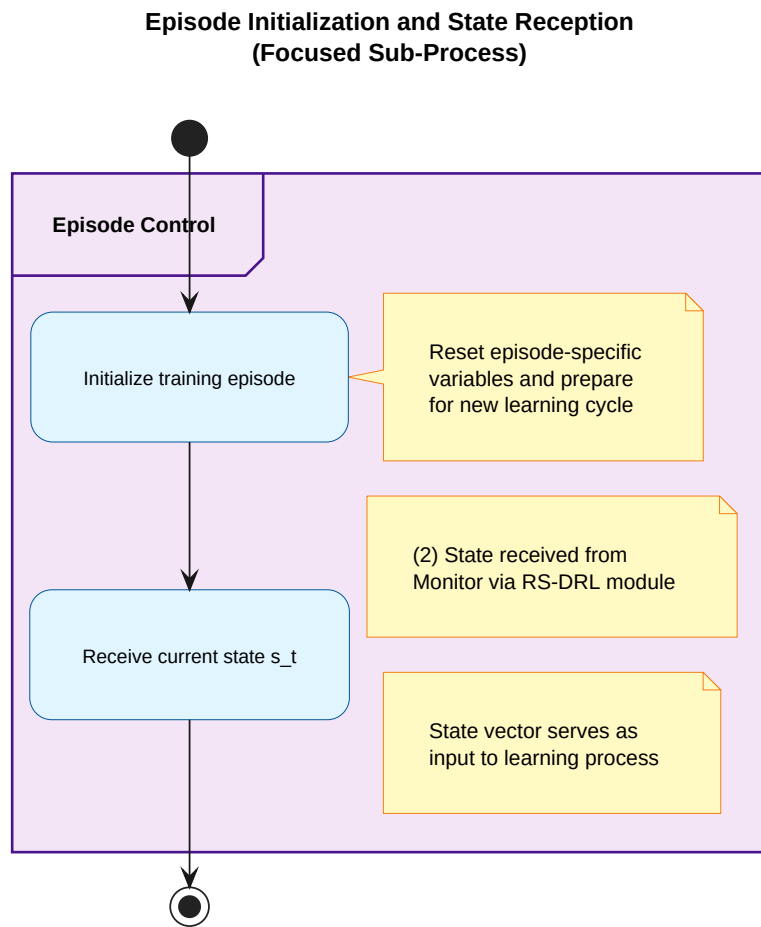


Figure 14: Experience collection during training episodes. This focused view shows episode initialization and interaction only. Complete learning workflow in Appendix C.

5.10.3 Novel Reward-Shaping Mechanism

The **reward-shaping mechanism** represents the key innovation of the RS-DRL approach. This mechanism:

- Selectively reshapes failed experiences
- Treats certain failures as optimistic successes
- Accelerates learning in heterogeneous environments

- Promotes policy generalization

The reward-shaping process operates during model training, transforming the experience distribution to facilitate faster convergence.

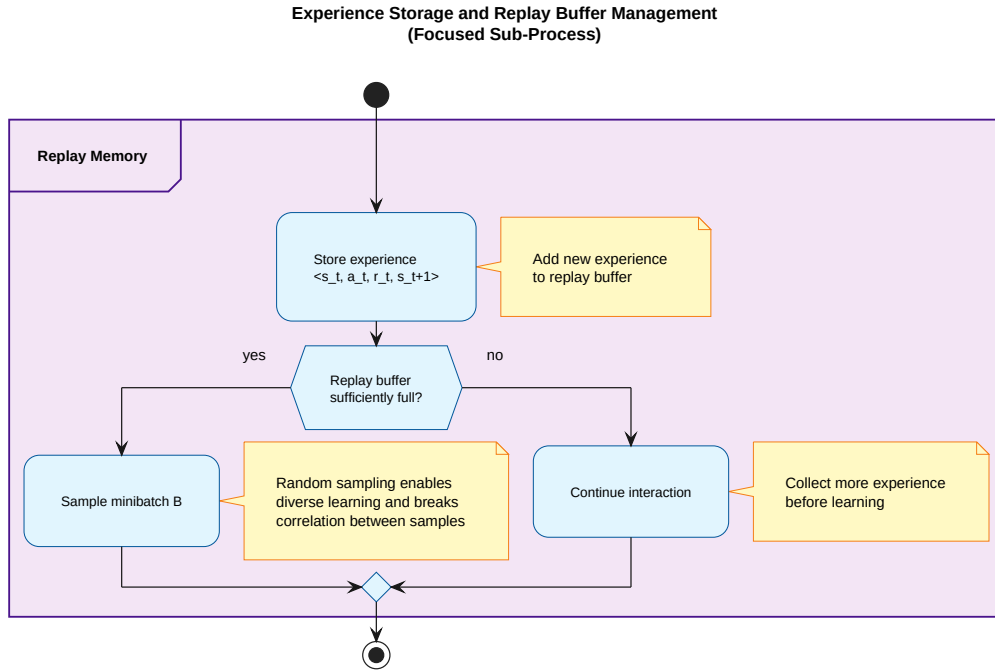


Figure 15: Experience replay and novel reward-shaping mechanism. This focused view highlights the key innovation. Complete learning workflow in Appendix C.

5.10.4 Model Training and Storage

Model training updates the deep Q-network parameters through gradient descent. The updated model is stored in the Knowledge repository (Stage 3), making it available for future runtime adaptation (Stage 6).

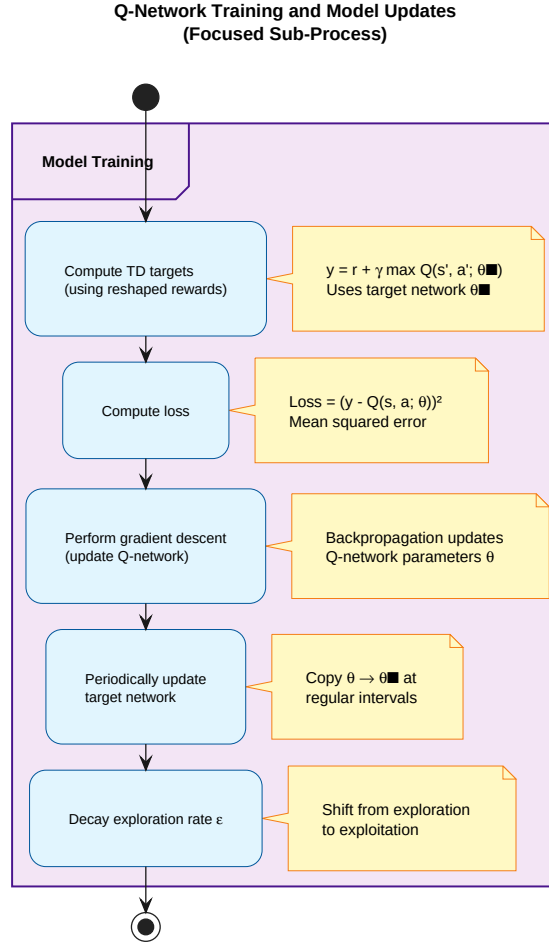


Figure 16: Q-network training and model update. This focused view shows policy improvement only. Complete learning workflow in Appendix C.

For the complete end-to-end learning workflow showing all steps integrated, see Appendix C.

5.11 Knowledge Management and Process Coordination

5.11.1 Knowledge as Shared System Memory

The Knowledge repository serves as shared system memory, storing trained models, adaptation options, and state history.

5.11.2 Read–Write Coordination Contracts

Knowledge coordination occurs through well-defined contracts:

- **When monitoring completes** (Stage 1): State vectors stored
- **When learning completes** (Stage 3): Trained models written
- **When planning begins** (Stage 6): Trained models read
- **When execution begins** (Stage 8): Adaptation specifications read

5.11.3 Coordination of Heterogeneous Processes

The Knowledge repository enables processes with different temporal characteristics to synchronize effectively without tight coupling.

5.12 Chapter Summary

5.12.1 Hierarchical Process Decomposition

This chapter presented the architecture and process model through progressive refinement:

1. System-level adaptation loop (black-box view)
2. MAPE-K conceptual mapping (classification lens)
3. Runtime monitoring and state construction (perception)
4. Analysis and triggering (control initiation)
5. Planning and prediction (decision intelligence)
6. Verification (uncertainty evaluation)
7. Execution (action)
8. Learning overview (policy improvement)
9. Knowledge management (coordination)

5.12.2 Architectural Contributions

Key architectural contributions include:

- Temporal separation of runtime decision-making and asynchronous learning
- Process-based traceability through nine-stage flow
- Verification as justification for learning-based decisions
- Knowledge as coordination mechanism for heterogeneous processes

5.12.3 Figure Organization Principle

This chapter employed **decomposed figures** aligned with section structure, where each figure focuses on one conceptual responsibility. This approach:

- Reduces cognitive load for examiners
- Aligns visual artifacts with textual modularity
- Preserves complete workflows in appendices
- Signals PhD-level presentation judgment

5.12.4 Transition to Evaluation

Having established the architectural foundation, the thesis proceeds to experimental evaluation (Chapter 6), demonstrating how the architectural design decisions enable effective adaptation under uncertainty.